SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models

26/04/10

SmoothQuant (PTQ)

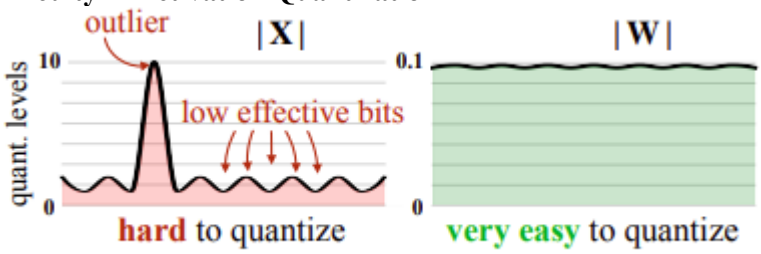
[기존 문제]

1. **LLM Quantization의 필요성**

   GPT-3 175B 모델을 단순 추론만 돌리는데 350GB VRAM 필요
   A100 GPU 80GB 5대 필요
   단순히 FP16을 INT8로만 바꿔도 memory 절반으로 줄어듦

   하지만, 저비트로 갈 수록 activation에서 특정 channel에 outlier ↑

2. **Difficulty in Activation Quantization**



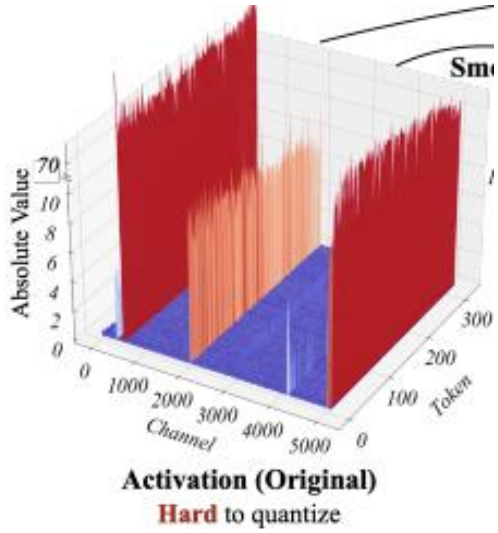
(a) Original

   W(가중치)는 고정된 값으로서 값이 전체 channel 간 균일한 반면, X(activation) 값은 추론 시 결정되는 dynamic 한 값이며, 실험적으로 **특정 channel 에서 심각한 outlier** 가 등장하는 것을 발견
   해당 값이 다른 다수의 값들과 100 배 이상 차이 나다 보니 scaling 에 의해 다른 channel 들 간의 미묘한 차이를 전부 collapse 하여 Quantization Error 가 커짐

- **LLM.int8() 논문: 그러면, 그냥 해당 outlier channel만 FP16으로 하거나 따로 처리하면 되지 않나?**
  => 이렇게 예외 처리를 하는 것은 **GPU, hardware 관점을 고려하지 않음**.
  GPU는 한꺼번에 많은 행렬 곱을 처리하는데 특화되어 있으며, 오히려 예외 처리로 인해 병목이 생겨 Full Precision 모델보다 느려짐.

[Activation Quantization Difficulty 추가 설명]

Activation Quantization이 어려운 이유는 특정 channel에서 심각한 outlier가 존재하기 때문이다. 추가로 관측한 사실은, 각 token들에 대해 특정 channel에서만 반복적으로 outlier가 발생한다는 것이다.

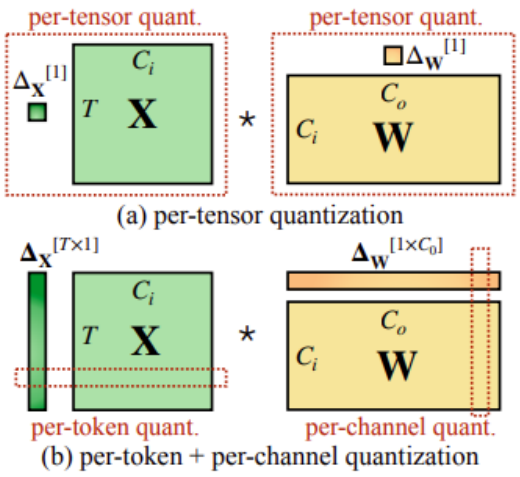


위의 그림과 같이 outlier가 특정 channel에 분포하는 것을 확인할 수 있다. 더 나아가, 단일 channel 내에서는 값의 variance가 작다.

즉, Intra-Channel의 variance는 작지만, Inter-Channel의 variance는 크다.

⇨ **Per-Channel activation quantization 하면 되는 거 아님?**

- **Per-Channel Activation Quantization이 안되는 이유 (LLM.int8() 추가 설명)**



위와 같이 **Y=XW** 에서 X는 [T(token#) x Ci(activation dim)] W는 [Ci x Co(output dim)]이다. Activation을 channel 별 scaling하기 위해서는 Ci 즉, inner dimension을 scaling해주어야 하는

데, 하드웨어 관점에서 이것이 불가.

외부 차원(C_o) 스케일링 (가능)

가중치의 각 출력 채널(j)마다 서로 다른 스케일값 $s_{W,j}$를 가진다고 가정해봅시다.

$$y_{tj} = \sum_{k=1}^{C_i} (x_{tk} \cdot (w_{kj} \cdot s_{W,j}))$$

여기서 s_{W,j}$는 시그마($\sum) 연산 동안 k값에 상관없이 **상수**입니다. 따라서 괄호 밖으로 쏙 빠집니다.

$$y_{tj} = s_{W,j} \cdot \left(\sum_{k=1}^{C_i} x_{tk} \cdot w_{kj} \right)$$

결론: 하드웨어는 괄호 안의 정수(INT8) 연산을 최대 속도로 다 끝낸 뒤, 마지막에 스케일($s_{W,j}$)만 딱 한 번 곱해주면 됩니다.

내부 차원(C_i) 스케일링 (불가능)

반면, 액티베이션의 입력 채널(k)마다 스케일 $s_{X,k}$가 다르다면 어떻게 될까요?

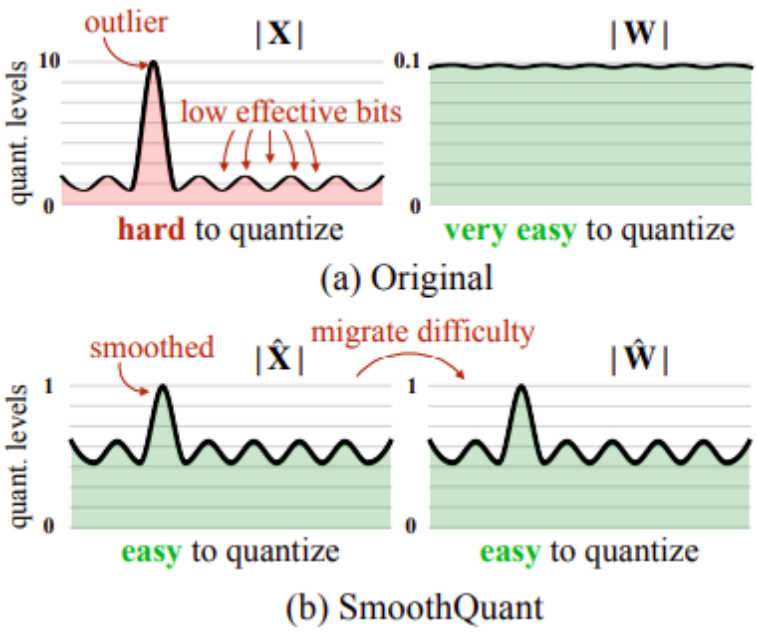
$$y_{tj} = \sum_{k=1}^{C_i} (x_{tk} \cdot s_{X,k} \cdot w_{kj})$$

이 경우 $s_{X,k}$는 k가 변함에 따라 계속 같이 변합니다. **시그마 밖으로 나갈 수가 없습니다.**

결론: 덧셈을 한 번 할 때마다 매번 스케일을 곱해야 하므로, "정수끼리 일단 다 더하고 나중에 변환한다"는 양자화의 대전제가 깨집니다.

즉, outer dimension인 weight의 channel에 대해서는 scale factor를 분리하여, 마지막에 한 번에 적용할 수 있지만, inner dimension인 activation의 channel은 이것이 불가능하기 때문에, hardware에 unfriendly하다. (가능은 하지만, 매우 느려진다.)

[SmoothQuant]



이에 SmoothQuant는 activation quantization difficulty를 weight quantization과 분담하는 방법론을 제시 (activation은 smoothing factor s 로 나누고, weight에는 s를 곱해서 output은 동일하지만, outlier 값을 좀 더 smooth하게 변환)

$$\mathbf{Y} = (\mathbf{X}\mathrm{diag}(\mathbf{s})^{-1}) \cdot (\mathrm{diag}(\mathbf{s})\mathbf{W}) = \hat{\mathbf{X}}\hat{\mathbf{W}}$$

위의 식과 같이 outlier가 큰 activation을 줄이기 위해 s로 나누고, 출력을 유지하기 위해 그만큼 s를 weight에 곱하여 activation의 quantization difficulty를 분담한다.

이를 Offline(Calibration set)에서 미리 계산해두어 extra scaling에 대해 overhead X

- **Smoothing Factor s**

  단순히 s를 max(X_j), 즉 activation의 max 값으로 smoothing하는 것은 오히려 weight에 quantization에 대한 부담을 전부 주기 때문에, 이를 적절히 나눠 가져야 함.

  즉, quantization difficulty를 적절히 나눠서 모두 quantization이 easy하도록!

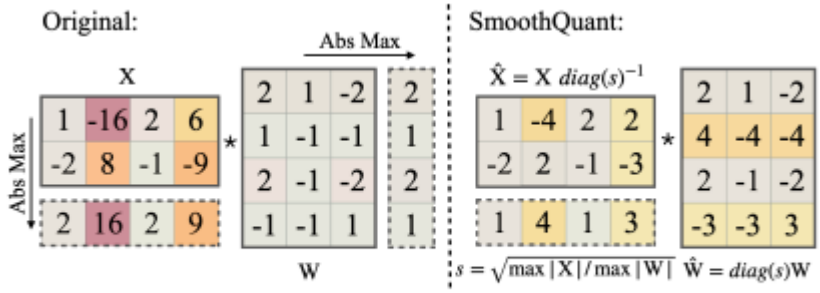


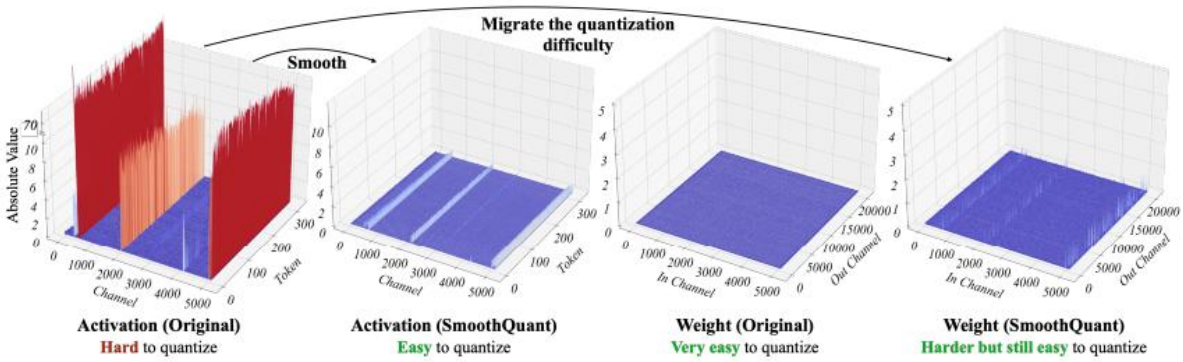
Figure 5: Main idea of SmoothQuant when α is 0.5. The smoothing factor s is obtained on calibration samples and the entire transformation is performed offline. At runtime, the activations are smooth without scaling.

Here we introduce a hyper-parameter, migration strength α, to control how much difficulty we want to migrate from activation to weights, using the following equation:

$$s_j = \max(|\mathbf{X}_j|)^{\alpha} / \max(|\mathbf{W}_j|)^{1-\alpha} \tag{4}$$

이에 smoothing factor를 activation과 weight의 최대값을 모두 고려하여 결정. Alpha는 hyper parameter 값이며, default 값은 0.5로 설정한다.

이를 통해 아래와 같이 activation의 분포가 smoothing 된다.



기존의 inter-channel 간 outlier에 의해 variance가 컸던 activation 값들이 전부 smoothing되어, 분포가 안정화된 것을 확인할 수 있다. Weight 또한, 비록 이전에 비해 smoothing factor에 의해 variance는 커졌지만, 여전히 전체 channel 간 분포가 smooth한 것을 확인할 수 있다.

이에 최종적으로 activation과 weight 모두 quantization 간 scale size에 따른 quantization error가 줄어든다.

[Experiments]

- **Activation Channel-wise Quantization**

Table 1: Among different activation quantization schemes, only per-channel quantization (Bondarenko et al., 2021) preserves the accuracy, but it is *not* compatible (marked in gray) with INT8 GEMM kernels. We report the average accuracy on WinoGrande, HellaSwag, PIQA, and LAMBADA.

Model size (OPT-)	6.7B	13B	30B	66B	175B
FP16	64.9%	65.6%	67.9%	69.5%	71.6%
INT8 per-tensor	39.9%	33.0%	32.8%	33.1%	32.3%
INT8 per-token	42.5%	33.0%	33.1%	32.9%	31.7%
INT8 per-channel	64.8%	65.6%	68.0%	69.4%	71.4%

앞서 말한 바와 같이, channel 별 activation quantization은 hardware에서 병목으로 작용하여 매우 느려지지만, 성능만 놓고 보았을 때 매우 좋다.

⇨ Activation에서 특정 channel의 outlier에 의한 collapse가 큰 문제임을 다시금 증명

- **Overall Benchmark 결과**

Table 2: Quantization setting of the baselines and SmoothQuant. All weight and activations use INT8 representations unless specified. For SmoothQuant, the efficiency **improves** from O1 to O3 (i.e., lower latency).

Method	Weight	Activation
W8A8	per-tensor	per-tensor dynamic
ZeroQuant	group-wise	per-token dynamic
`LLM.int8()`	per-channel	per-token dynamic+FP16
Outlier Suppression	per-tensor	per-tensor static
SmoothQuant-O1	per-tensor	per-token dynamic
SmoothQuant-O2	per-tensor	per-tensor dynamic
SmoothQuant-O3	per-tensor	per-tensor static

다음의 baseline을 가지고 실험을 진행.

O1과 O2는 scale size를 test time에 dynamic하게 계산하며, O3는 calibration set을 바탕으로 static하게 저장. (따라서, O3가 좀 더 빠름)

Table 3: SmoothQuant maintains the accuracy of OPT-175B model after INT8 quantization, even with the most aggressive and most efficient O3 setting (Table 2). We extensively benchmark the performance on 7 zero-shot benchmarks (by reporting the average accuracy) and 1 language modeling benchmark (perplexity). *For ZeroQuant, we also tried leaving the input activation of self-attention in FP16 and quantizing the rest to INT8, which is their solution to the GPT-NeoX-20B. But this does not solve the accuracy degradation of OPT-175B.

OPT-175B	LAMBADA	HellaSwag	PIQA	WinoGrande	OpenBookQA	RTE	COPA	**Average↑**	**WikiText↓**
FP16	74.7%	59.3%	79.7%	72.6%	34.0%	59.9%	88.0%	66.9%	10.99
W8A8	0.0%	25.6%	53.4%	50.3%	14.0%	49.5%	56.0%	35.5%	93080
ZeroQuant	0.0%*	26.0%	51.7%	49.3%	17.8%	50.9%	55.0%	35.8%	84648
`LLM.int8()`	74.7%	59.2%	79.7%	72.1%	34.2%	60.3%	87.0%	66.7%	11.10
Outlier Suppression	0.00%	25.8%	52.5%	48.6%	16.6%	53.4%	55.0%	36.0%	96151
SmoothQuant-O1	74.7%	59.2%	79.7%	71.2%	33.4%	58.1%	89.0%	66.5%	11.11
SmoothQuant-O2	75.0%	59.0%	79.2%	71.2%	33.0%	59.6%	88.0%	66.4%	11.14
SmoothQuant-O3	74.6%	58.9%	79.7%	71.2%	33.4%	59.9%	90.0%	66.8%	11.17

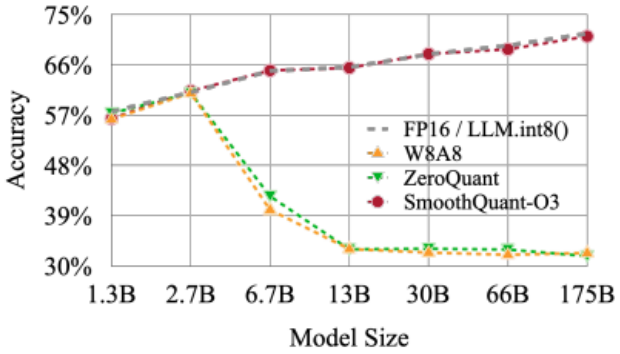


Figure 7: SmoothQuant-O3 (the most efficient setting, defined in Table 2) preserves the accuracy of OPT models across different scales when quantized to INT8. LLM.int8() requires mixed precision and suffers from slowing down.

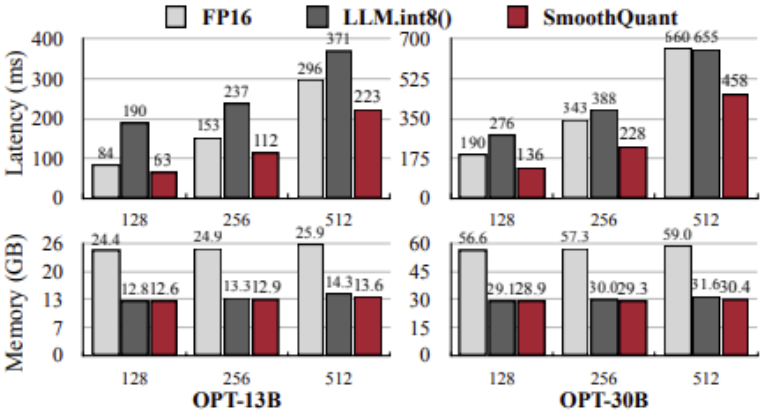


Figure 8: The PyTorch implementation of SmoothQuant-O3 achieves up to **1.51**× speedup and **1.96**× memory saving for OPT models on a single NVIDIA A100-80GB GPU, while `LLM.int8()` slows down the inference in most cases.

SmoothQuant가 LLM.int8()과 달리 only INT8로 SOTA 성능과 memory size, 속도 모두 개선.

Figure 8.에서 볼 수 있듯이, LLM.int8()은 오히려 Latency에 있어 느려짐.

[논문 리뷰 과정 중 참고 자료]

diag로 activation을 나누는건 되고, 왜 X의 C_i에 대해 스케일링하는건 불가능해>

아주 예리하고 훌륭한 질문입니다! 수식을 보면 분명 둘 다 액티베이션(X)의 채널(C_i) 차원을 건드리는 것 같은데, 왜 하나는 되고 하나는 안 되는지 헷갈리실 수 있습니다.

핵심부터 말씀드리면, `diag(s)` **를 통해 액티베이션을 나누는 작업은 추론을 시작하기 전(Offline)에 '이전 레이어의 가중치'에 미리 병합(Fusion)해버리기 때문에 런타임 연산이 아예 발생하지 않아서 가능한 것**이고, **양자화 스케일(Δ)을 C_i에 적용하는 것은 초고속 INT8 연산기(GEMM)가 돌아가는 '중간(Runtime)'에 개입해야 해서 불가능**한 것입니다.

이 두 가지 상황의 차이를 하드웨어와 연산 시점 관점에서 명확히 비교해 드리겠습니다.

1. `diag(s)` 로 X를 나누는 것이 가능한 이유: 오프라인 병합 (Offline Fusion) SmoothQuant에서 액티베이션을 평탄화하기 위해 사용하는 수식 $\hat{X} = X\text{diag}(s)^{-1}$는 런타임에 직접 나누기 연산을 수행하는 것이 아닙니다 [1] [2].

- 액티베이션 X는 허공에서 나타나는 것이 아니라, **항상 '이전 레이어(예: LayerNorm이나 이전 선형 레이어)'를 통과해서 나온 결과물**입니다 [2].
- 따라서 연구진은 런타임에 X를 스케일링하는 대신, **모델을 서비스하기 전(Offline)에 이전 레이어의 파라미터(예: LayerNorm의 가중치)를 미리 s로 나누어 저장**해 둡니다 [2].
- 이렇게 해두면, 런타임에는 아무런 추가 스케일링 연산(kernel call overhead)을 할 필요 없이 이전 레이어를 통과하고 나온 결과물이 **자연스럽게 이미 s로 나누어진 상태(평탄화된 $\hat{X}$)로 출력**됩니다 [2]. 즉, 하드웨어 입장에서는 그냥 평소 하던 대로 연산했을 뿐인데 수학적 트릭에 의해 스케일링이 완료된 것입니다.

2. X의 C_i 차원에 대해 양자화 스케일링(Δ)을 하는 것이 불가능한 이유: 런타임 내부 차원 개입 반면, SmoothQuant의 트릭을 쓰지 않고 액티베이션 자체를 채널(C_i)별로 양자화(Per-channel activation quantization)하려고 하면 완전히 다른 문제가 발생합니다 [3] [4].

- 행렬 곱셈 커널(Tensor Core MMAs)은 아주 빠른 속도(High throughput)로 끊임없이 정수형(INT8) 덧셈과 곱셈을 반복해야 합니다 [4].
- 이때 C_i는 두 행렬이 맞닿아 곱해지면서 소거되는 **내부 차원(Inner dimension)**입니다 [4] [5]. 특정 결과값을 구하기 위해 C_i 방향으로 이동하며 덧셈을 누적해야 하는데, 만약 C_i 채널마다 양자화 스케일(눈금 간격, Δ)이 다르다면 어떻게 될까요?
- 하드웨어는 순수 INT8 덧셈을 멈추고, 매번 데이터 형변환(Conversions)을 하거나 느린 부동소수점 연산(CUDA Core FMAs)을 끼워 넣어 스케일을 맞춰주어야 합니다 [4].
- 즉, **초고속으로 달려야 할 런타임 파이프라인 중간중간에 처리량이 낮은 명령어들이 강제로 삽입**되어 하드웨어 가속의 의미가 완전히 파괴되기 때문에 불가능한 것입니다 [4].

결론 요약: SmoothQuant가 C_i 차원에 적용하는 s (스무딩 팩터)는 **모델 파라미터에 미리 녹여내어 (Offline fusion) 런타임 연산을 아예 없애버렸기 때문에 가능**한 것입니다 [2]. 반면, 양자화 스텝 사이즈 Δ를 C_i에 적용하려는 시도는 **절대 끊기면 안 되는 INT8 덧셈 연산(Runtime)의 한가운데를 비집고 들어가야 하므로 하드웨어 구조상 불가능**한 것입니다 [4].

다시 말해, weight는 outer channel이라 되고, activation은 inner channel이라 안된느거지?

네, 정확하게 이해하셨습니다!

고속 하드웨어 가속기(GEMM 커널 등)를 효율적으로 활용하기 위한 양자화 스케일링은 행렬 곱셈의 **외부 차원(Outer dimensions)에서만 가능하며, 내부 차원(Inner dimension)에서는 불가능**합니다 [1] [2].

- **가중치(Weight) = 외부 차원 (적용 가능):** 가중치의 채널 단위 양자화는 행렬 곱셈의 외부 차원에 해당하는 **출력 채널(C_o)**을 기준으로 수행됩니다 [1] [2]. 외부 차원에 대한 스케일링은 행렬 곱셈 연산이 모두 끝난 직후에 바깥쪽에서 일괄적으로 적용할 수 있으므로, 하드웨어의 고속 파이프라인 흐름을 전혀 방해하지 않고 효율적으로 작동합니다 [2].
- **액티베이션(Activation) = 내부 차원 (적용 불가):** 반면 액티베이션을 채널 단위로 양자화하려면, 두 행렬이 맞닿아 곱해지면서 소거되는 내부 차원인 **입력 채널(C_i)**에 각각 다른 스케일링을 적용해야 합니다 [1]. 텐서 코어(Tensor Core)와 같은 하드웨어 가속기는 연속적인 고속 연산을 처리하도록 설계되어 있는데, 내부 차원에 스케일링을 하려면 연산 중간중간에 데이터 형변환과 같은 처리량이 낮은(lower throughput) 명령어가 삽입되어야 하므로 하드웨어가 이를 허용(tolerate)하지 않습니다 [2].

결론적으로 질문하신 내용처럼 **가중치는 외부 차원(Outer dimension)에 해당하여 하드웨어 친화적인 스케일링이 가능하고, 액티베이션의 채널은 내부 차원(Inner dimension)에 해당하여 불가능**한 것이 핵심 원리입니다 [1] [2].

메모에 저장

2. 하드웨어 직관: "환전소와 컨베이어 벨트"

Tensor Core를 아주 빠른 **컨베이어 벨트(MMA 파이프라인)**라고 생각해 보세요. 이 벨트는 오직 '정수(Integer) 박스'만 운반할 수 있습니다.

- **가중치 채널 스케일링(Outer):**
  컨베이어 벨트가 끝나는 지점에 **'환전소(Scaling Unit)'**가 있는 구조입니다. 벨트 위에서는 정수들끼리 신나게 더해지고(Accumulate), 마지막에 벨트를 빠져나온 완성품에만 "이건 달러($) 기준이야", "이건 엔화(¥) 기준이야"라고 라벨을 붙여주는 꼴입니다. 흐름이 끊기지 않습니다.
- **액티베이션 채널 스케일링(Inner):**
  벨트 중간중간에 환전소가 있어야 하는 구조입니다. "정수 A랑 B를 더하려는데, A는 달러고 B는 유로네? 잠깐 멈춰봐, 환전하고 더하자." 하는 순간 컨베이어 벨트는 멈춰 서게 됩니다. GPU 입장에서는 최악의 병목 현상이 발생하는 거죠.

3. 시각적 구조 비교

행렬의 관점에서 스케일링이 적용되는 위치를 시각화하면 다음과 같습니다.

구분	내부 차원 (Ci) 스케일링	외부 차원 (Co) 스케일링
적용 위치	행렬이 맞닿는 **안쪽**	결과 행렬의 **바깥쪽**
수학적 의미	시그마 내부 연산 (분리 불가)	시그마 외부 연산 (분리 가능)
하드웨어 부하	**매 곱셈/덧셈마다** 연산 필요	**최종 결과값에만** 1회 연산 필요
비유	요리 중간에 간 맞추기 (계속 멈춤)	요리 다 하고 서빙 직전 데코 (빠름)

Sheets로 내보내기